

12. Bölüm

İstisna Yönetimi

12.1 Yapısal Programlamada Hata Yönetimi

Yapısal programlamada sorun olan, hata ayıklama kodu ile işlevsel kodun iç içe olması durumudur. Nesne yönelimli programlamanın getirdiği yenilikle bu sorun tarihe karışmıştır. Yapısal programlamada hatalı bir duruma yol açmamak ya da hatalı bir durumla karşılaşıldığında programdan çıkılır. Aşağıda bir C program örneği verilmiştir;

```
#include <stdio.h>
int faktoriyel(int sayi) {
    if (sayi<0) return -1;
    if(sayi <= 1) return 1;
    return sayi * faktoriyel(sayi - 1);
}
int main() {
    int okunan,deger,hata;
    do {
        printf("Bir Sayı Giriniz:");
        scanf("%d",&okunan);
        deger=faktoriyel(okunan);
        printf("%d nin faktöriyeli: %d\n", okunan, deger);
        if (deger==-1) {
            hata=100;
            break; // exit(100);
        }
    } while (okunan!=0);
    return hata;
}
/* Programın Çıktısı:
Bir Sayı Giriniz:-1
-1 nin faktöriyeli: -1
*/
```

Bu program örneğinde `faktoriyel` fonksiyonu hata durumunda `-1` geri döndürmektedir. Ancak gerçek hayatta bu fonksiyonun böyle bir değer döndürmesi mümkün değildir.

12.2 İstisna Yönetimi

Nesne yönelimli programlamada (OOP) ise **istisna** (**exception**), bir programın yürütülmesi sırasında ortaya çıkan beklenmeyen bir sorundur. İstisna, çalışma sırasında (**run-time**) oluşur. Bu tür beklenmeyen **özel durumlarda** imal edilen istisna nesneleri (**exception object**) vardır. İstisna iki türlü ortaya çıkar;

1. **Eşzamanlı** (**synchronous**): Giriş verilerindeki bir hata nedeniyle bir şeylerin ters gitmesi veya programın çalıştığı mevcut veri türünü işleme durumunda oluşan istisnalar (örneğin bir sayıyı sıfıra bölmek).
2. **Zaman Uyumsuz** (**asynchronous**): Disk arızası, klavye kesintileri vb. gibi yazılan programın kontrolü dışındaki istisnalar.

İstisnai durumları yönetmek için `throw`, `try` ve `catch` anahtar kelimeleri kullanılır. İstisnaları işlemek için sözde kod aşağıda verilmiştir;

```
try {
    /*
    ...
    İstisnai Durumun Ortaya Çıkabileceği Kod...
    ...
    */
    throw SomeExceptionType("İstisnai Durum Açıklaması");
    /* throw an exception: istisnai bir duruma düşme */
```

```

} catch( ExceptionName1 e1 ) {
    /* catch bloğu try bloğundaki istisnayı yakalamak ve gerekli işlemi yapmak için kullanılır. */
}

```

Bu sözde kod şu şekilde açıklanabilir;

- ◊ **try talimatı:** İçerisinde istisnai bir duruma düşülebileceğini tahmin ettiğimiz bir kod bloğunu temsil eder. Bu bloğu bir veya daha fazla **catch** bloğu takip eder.
- ◊ **catch talimatı:** **try** bloğunda bir istisnai duruma düşüldüğünde bu istisnayı (özel durumu) işlemek için yazılacak kod, **catch** bloğunun içine yazılır.
- ◊ **İstisnai duruma düşme:** Bir istisnai duruma **throw** anahtar sözcüğü kullanılarak da düşülebilir. İstisna nesneleri imal edilince bir havuza atılıyormuş gibi **istisna nesneleri fırlatılır (throw an exception)**.
- ◊ **İstisna sonrası süreç:** Bir program bir **throw** ifadesiyle karşılaştığında hemen içinde bulunduğu **try** bloğu dışına çıkılır ve ilgili istisnayı işlemek için eşleşen bir **catch** bloğu bulmaya başlar. Bulduğunda istisna havuzdan **bir istisna nesnesi yakalanmış (catch an exception)** olur.
- ◊ Bir **try** bloğu içinde birden fazla **throw** ifadesi bulunabilir.

Bu yeni bakış açısıyla 12.1 başlığındaki yapısal programlama faktöriyel örneği aşağıdaki gibi yazabiliriz;

```

using System;
namespace HataYonetimi
{
    class Program
    {
        // Faktöriyel fonksiyonu
        static int Faktoriyel(int sayi)
        {
            if (sayi < 0)
            {
                throw new ArgumentException("Negatif sayıların faktöriyeli hesaplanamaz!");
            }

            if (sayi <= 1) return 1;
            return sayi * Faktoriyel(sayi - 1);
        }
        //Ana Program
        static void Main()
        {
            int okunan;
            while (true)
            {
                Console.WriteLine("Bir Sayı Giriniz (Çıkış için 0): ");
                if (!int.TryParse(Console.ReadLine(), out okunan)) continue;
                if (okunan == 0) break;
                try
                {
                    // Hata riski olan kod bloğu
                    int sonuc = Faktoriyel(okunan);
                    Console.WriteLine($"{okunan} sayısının faktöriyeli: {sonuc}");
                }
                catch (ArgumentException e) // Özel bir hata türünü yakalıyoruz
                {
                    // e.Message fırlatılan hata metnini içerir.
                    Console.WriteLine($"HATA: {e.Message}");
                    // return -1; // İstenirse program sonlandırılabilir
                }
                catch (Exception e) // Beklenmedik tüm hataları yakalamak için (Genel havuz)
                {
                    Console.WriteLine($"Beklenmedik bir hata oluştu: {e.Message}");
                }
            }
        }
    }
}

```

İstisnalar Neden İşlenmeli?

- ◊ Hata işleme kodunun işlevsel koddan (iş mantığını içeren koddan) ayrılması için.
- ◊ Yöntemler yalnızca seçtikleri istisnaları işleyebilir. Bir yöntemde birden çok istisnai duruma düşülebilir. Ancak bunlardan bazılarını `catch` bloğunda işlemeyi seçebilir. Geri kalanlarını yöntemi çağıran yere bırakabilir.
- ◊ Hata Türlerinin Gruplanması: C# dilinde yeni bir istisna sınıfı tanımlanabileceği gibi mevcut istisna sınıflarından da istisna nesnesi imal edilebilir.

Aşağıda dizi sınırları ile ilgili bir program örneği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
int[] array = {10, 20, 30 };
try
{
    array[4]=40;
}
catch (Exception e) //tüm istisna nesneleri yakalanabilir!
{
    Console.WriteLine("Dizi indisi sınırların dışında!");
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine($"Hata Verisi:{e.Data}");
    Console.WriteLine($"Hataya Sebep Olan İç Hata:{e.InnerException}");
}
/* Programın Çıktısı:
Dizi indisi sınırların dışında!
Hata Mesajı:Index was outside the bounds of the array.
Hata Verisi:System.Collections.ListDictionaryInternal
Hataya Sebep Olan İç Hata:

...Program finished with exit code 0
*/
```

12.3 Standart İstisna Tipleri

Aşağıda sıfıra bölme hatasını işleyen bir program örneği verilmiştir. Örnekte standart istisna tiplerinden biri olan `DivideByZeroException` istisnası imal edilmiştir.

```
using System;
namespace HataYonetimi
{
    class Program
    {
        static void Main()
        {
            Console.WriteLine("Burası her zaman icra edilir.");
            try
            {
                // try BLOĞU: İstisna oluşma ihtimali olan yer
                int bolunen = 10;
                int bolen = 0;
                if (bolen == 0)
                {
                    // Manuel olarak hazır bir hata sınıfı fırlatıyoruz
                    throw new DivideByZeroException("Sıfıra Bölmeye İzin Verilmez!");
                    // throw satırından sonraki kodlar asla çalışmaz
                    Console.WriteLine("Burası hiçbir zaman icra edilmez.");
                }
                int sonuc = bolunen / bolen;
                Console.WriteLine($"Bölme Sonucu: {sonuc}");
            }
            catch (DivideByZeroException e) // catch BLOĞU: Sadece sıfıra bölme hatası
            {
                // yakalandığında çalışır
            }
        }
    }
}
```

```

    {
        Console.WriteLine($"İstisna Yakalandı: {e.Message}");
    }
    catch (Exception e) //Varsayılan Catch
    {
        // C++'taki catch (...) bloğunun karşılığıdır.
        // Yukarıdaki özel tiplere uymayan tüm hatalar burada yakalanır.
        Console.WriteLine($"Genel bir hata oluştu: {e.Message}");
    }
    Console.WriteLine("Burası da her zaman icra edilir.");
}
}
}

```

Yukarıdaki örnekte `try` bloğunda izin verilmeyen bölme işleminde istisnai bir durum fark ediliyor ve istisna nesnesi imal ediliyor ve `throw` ile fırlatılıyor (`throw an exception`). `catch` bloğunda ise (istisna nesneleri havuzundan) **istisna nesnesi yakalanarak** (`catch an exception`) işleniyor. Örnekte standart tipte istisna nesnesi (`DivideByZeroException`) kullanılmıştır. Bunun gibi kullanılacak istisna nesnelerinin listesi Tablo 12.1’de verilmiştir;

İstisna Tipi	Açıklama
<code>ArithmeticException</code>	Aritmetik işlemler sırasında oluşan hatalar için temel sınıftır (<code>DivideByZeroException</code> ve <code>OverflowException</code> gibi).
<code>ArrayTypeMismatchException</code>	Bir diziye, dizinin türüyle uyumsuz bir öge yerleştirilmeye çalışıldığında fırlatılır.
<code>DivideByZeroException</code>	Bir tamsayı değerini sıfıra bölme girişimi gerçekleştiğinde fırlatılır.
<code>IndexOutOfRangeException</code>	Bir dizinin sınırları dışındaki (negatif veya boyuttan büyük) bir indekse erişilmeye çalışıldığında fırlatılır.
<code>InvalidCastException</code>	Zorlama (<code>casting</code>) sırasında geçersiz bir tip dönüşümü işlemi çalışma zamanında başarısız olduğunda fırlatılır.
<code>NullReferenceException</code>	Değeri <code>null</code> olan bir nesne referansı üzerinden bir üyeye (yöntem, özellik vb.) erişilmeye çalışıldığında fırlatılır.
<code>OutOfMemoryException</code>	Yeni bir nesne oluşturmak (<code>new</code>) için yeterli sistem belleği kalmadığında fırlatılır.
<code>OverflowException</code>	<code>checked</code> blokları içinde yapılan aritmetik işlemlerin sonucunun, veri tipinin sınırlarını aşması durumunda fırlatılır.
<code>StackOverflowException</code>	Yürütme yığını (<code>stack</code>), genellikle sonsuz özyineleme (<code>recursion</code>) nedeniyle tükendiğinde fırlatılır.
<code>TypeInitializationException</code>	Statik bir yapıcı (<code>static constructor</code>) hata verdiğinde veya statik bir alan (<code>field</code>) başlatılamadığında fırlatılır.

Tablo 12.1: Yaygın Sistem İstisnaları

Bir `try` bloğunda (istisna havuzuna) birden fazla istisna nesnesi fırlatılabilir. Ama tüm `catch` talimatlarıyla bu istisnalar yakalanmayabilir. Ayrıca tüm istisna nesnelerin tipleri de bilinmeyebilir. Bu durumda `catch (Exception e)` şeklinde kodlanan varsayılan `catch` kullanılır.

Sistem tarafından istisnaların yakalanması yerine istisnaları biz `throw` anahtar kelimesiyle yeni bir istisna nesnesi imal edip yönetebiliriz. Anı örnek bu durumda aşağıdaki gibi yazılabilir;

```

int[] array = {10, 20, 30 };
try
{
    int indis = 4;
    if (indis >= array.Length) // İstisnai durum
    {
        //İstisna nesnesi oluşturulup (istisna havuzuna) fırlatılıyor
        throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
    }
    array[indis]=40;
}
catch (Exception e)
{

```

```

Console.WriteLine($"Hata Mesajı:{e.Message}");
Console.WriteLine($"Hata Verisi:{e.Data}");
Console.WriteLine($"Hataya Sebep Olan İç Hata:{e.InnerException}");
}

```

Bir `try` bloğunda birden fazla istisna nesnesi (istisna havuzuna) fırlatılabilir; Aşağıdaki örnekte `try` bloğunda birden fazla istisnai nesnesi fırlatılmıştır.

```

int[] array = {10, 20, 30 };
try
{
    int indis = 2;
    if (indis >= array.Length) //koşul sağlanmaz!
    {
        throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
    }
    array[2]=40;
    int bolen=0, bolunen=100;
    if (bolen == 0)
    {
        throw new DivideByZeroException("Bölme işlemi sıfıra bölünemez!");
    }
}
catch (IndexOutOfRangeException e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("İndis Hatası Olunca Neler Yapılmalı? Burada Yazılır.");
}
catch (DivideByZeroException e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("Sıfıra Bölme Olunca Neler Yapılmalı? Burada Yazılır.");
}
/*Program Çıktısı:
Hata Mesajı:Bölme işlemi sıfıra bölünemez!
Sıfıra Bölme Olunca Neler Yapılmalı? Burada Yazılır.
*/

```

Yukarıdaki programda ilk karşılaşılan özel durum sıfıra bölme durumudur ve `throw` ile (istisna havuzuna) `DivideByZeroException` nesnesi fırlatılır. Bu durumda `catch` bloklarına sırasıyla bakılır ilgili özel durum nesnesini işleyecek olan `catch` bloğu icra edilir.

Aşağıda `Kişi` sınıfta `Yaş` özelliğine ilişkin istisna içeren bir kod örneği verilmiştir;

```

//ANA PROGRAM
Kişi ali = new Kişi();
try
{
    // 1. Geçerli bir değer atama
    ali.Yaş = 12;
    Console.WriteLine($"Güncel Yaş: {ali.Yaş}");
    // 2. İstisna yaratacak değer (Hata fırlatıldığı an try bloğu terk edilir)
    ali.Yaş = -1;
    // --- BU SATIRDAN SONRASI İCRA EDİLMEZ ---
    Console.WriteLine($"Bu satıra ulaşılamayacak: {ali.Yaş}");
}
catch (ArgumentOutOfRangeException ex)
{
    Console.WriteLine("İstisnai Bir Durum Yakalandı!");
    // ex.ParamName hataya sebep olan parametreyi, ex.Message ise açıklamayı verir.
    Console.WriteLine($"Hata Mesajı: {ex.Message}");
}
catch (Exception ex)
{
    Console.WriteLine($"Beklenmedik Hata: {ex.Message}");
}

```

```

}

//SINIF
public class Kişi
{
    private int _yaş; // Field (Alan)
    public Kişi()
    {
        _yaş = 15; // Varsayılan değer
    }
    public int Yaş
    {
        get { return _yaş; }
        set
        {
            if (value < 0)
                throw new ArgumentOutOfRangeException(nameof(Yaş), "Geçersiz Yaş Değeri: < 0!");
            else if (value > 120)
                throw new ArgumentOutOfRangeException(nameof(Yaş), "Çok Büyük Yaş Değeri: > 120!");
            _yaş = value;
        }
    }
}

```

12.4 İstisna Filtreleri

Bazı durumlarda birden fazla istisna tipini tek bir `catch` ile yakalamak isteriz. Bu durumda `when` süzgeci kullanılabilir;

```

int[] array = { 10, 20, 30 };
try
{
    int indis = 2;
    if (indis >= array.Length) //koşul sağlanmaz!
    {
        throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
    }
    array[2] = 40;
    int bolen = 0, bolunen = 100;
    if (bolen == 0)
    {
        throw new DivideByZeroException("Bölme işlemi sıfıra bölünemez!");
    }
}
catch (Exception e) when (e is IndexOutOfRangeException || e is DivideByZeroException)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("Hata Olunca Neler Yapılmalı? Burada Yazılır.");
}

```

12.5 Kullanıcı Tanımlı İstisnalar

Kendi istisna tipimizi oluşturmak için `Exception` sınıfından türemiş bir sınıf yazarız. Aşağıda `Kişi` sınıfta yaş özelliğine ilişkin istisna içeren bir kod örneği verilmiştir;

```

using System.Diagnostics; // StackTrace sınıfı için gerekli
int[] array = { 10, 20, 30 };
try
{
    int indis = 4;
    if (indis >= array.Length)
    {
        //throw new İndisİstisnası(); //Bu satır yada aşağıdaki

```

```

        string currentFile = new StackTrace(true).GetFrame(0).GetFileName();
        int currentLine = new StackTrace(true).GetFrame(0).GetFileLineNumber();
        throw new Dosyaİsimliİndisİstisnası(currentFile, currentLine);
    }
    array[indis] = 40;
}
catch (İndisİstisnası e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("İndis Hatası Olunca Neler Yapılmalı? Burada Yazılır.");
}
catch (Dosyaİsimliİndisİstisnası e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    // Hata mesajı: D:\CSharp\Samples\Program.cs Dosyası 12 Satırında İndis Hatası!
    Console.WriteLine($"Hata Mesajı:{e.FileName}");
    // Hata mesajı: D:\CSharp\Samples\Program.cs
    Console.WriteLine($"Hata Mesajı:{e.LineNumber}");
    // Hata mesajı: 12
}

public class İndisİstisnası : Exception
{
    public İndisİstisnası() : base("Dizide olmayan bir elemana indis belirlendi!")
    { }
}

public class Dosyaİsimliİndisİstisnası : Exception
{
    public Dosyaİsimliİndisİstisnası(string fileName, int lineNumber) :
    base($"{fileName} Dosyası {lineNumber} Satırında İndis Hatası!")
    {
        FileName = fileName;
        LineNumber = lineNumber;
    }
    public string FileName { get; private set; }
    public int LineNumber { get; private set; }
}

```

12.6 finally Bloğu

finally bloğu, bir **try** bloğunda gerçekleştirilen eylemleri temizlemenizi sağlar. Varsa, **finally** bloğu en son bloktur ve eşleşen **catch** bloklardan sonra icra edilir. Özel bir durum/istisna oluşturma veya özel durum/istisna tipiyle eşleşen bir **catch** blok bulunması fark etmeksizin **finally** bloğu her zaman çalışır. Ayrıca **finally** bloğu, **try** içinde **return** olsa dahi mutlaka çalıştırılır. Sözde kod aşağıda verilmiştir;

```

try
{
    /* istisna olabilecek kod burada yazılır. */
}
catch (Exception e)
{
    /* istisnayı yönetme kodu buraya yazılır. */
}
finally
{
    /* Bir istisna atılmış/yakalanmış olsun veya olmasın, yürütülecek kod buraya yazılır. */
}

```

Burada **catch** bloğu programcı tarafından kodlanmayabilir. Dosyalarla ilgili bir örnek aşağıda verilmiştir;

```

FileStream f = null;
try
{
    f = File.OpenRead("file.txt");
}

```

```

}
finally
{
    f?.Close(); /* f null olabilir, bu nedenle null koşullu işlecini kullanın.
    Çünkü dosya bir şekilde açılmamış olabilir. */
}

```

12.7 İstisnayı Tekrar Fırlatma

Bazı durumlarda istisna nesnesinin yakalanıp `catch` bloğunda işlenmesi sırasında mevcut istisna nesnesini `throw` ile (istisna havuzuna) **tekrar fırlatmak** (**re-throw**) gerekir.

```

try
{
    //...
}
catch (Exception ex)
{
    //...
    throw;
    //yada throw ex;
    //derleyiciye icra akışının asla try bloğundan ayrılmadığını bildirmek için yapılır.
}

```

`throw ex;` yığın izini sıfırlarken, `throw;` orijinal hatanın nerede oluştuğu bilgisini korur. Bu, kurumsal yazılımlarda hata ayıklama için hayati önemdedir. Katmanlı mimaride bir katmandan diğerine istisnaları iletmek için bu yöntem kullanılır. Genellikle iç katmanlarda hatanın gösterileceği bir kullanıcı ara yüzü bulunmaz.

12.8 Yığın Geri Sarma

Program akışı sırasında bir istisna (**exception**) fırlatıldığında, .NET çalışma zamanı (CLR) mevcut yöntemden başlayarak yukarıya doğru (çağrı hiyerarşisinde) uygun bir `catch` bloğu aramaya başlar. Bu arama sürecine ve bu sırada yerel değişkenlerin bellekten temizlenmesine **yığın geri sarma** (**stack unwinding**) denir.

İstisna Fırlatıldığında İzlenecek Adımlar

- ♦ **Arama:** CLR, hatanın oluştuğu metotta bir `try-catch` bloğu olup olmadığına bakar.
- ♦ **Geri Sarma:** Eğer bulamazsa, mevcut metodu sonlandırır ve onu çağıran bir üst metoda geçer. Bu sırada o metodun yığındaki (**stack**) yerel değişkenleri temizlenir.
- ♦ **Sonlandırma:** Her bir adımda, eğer varsa `finally` blokları icra edilir.
- ♦ **Sonuç:** Uygun bir `catch` bloğu bulunana kadar bu işlem "yığını geri sararak" devam eder. Eğer hiçbir yerde yakalanmazsa uygulama çöker (**unhandled exception**).

Aşağıdaki örnekte, hatanın en derinden en yukarıya nasıl tırmandığını görebilirsiniz:

```

using System;
class Program
{
    static void Main()
    {
        try {
            AYöntemi();
        }
        catch (Exception ex) {
            Console.WriteLine($"Hata Main'de Yakalandı: {ex.Message}");
            // StackTrace sayesinde hatanın hangi yollardan geçtiğini görebiliriz.
            Console.WriteLine(ex.StackTrace);
        }
    }
    static void AYöntemi() => BYöntemi();
    static void BYöntemi()
    {
        Console.WriteLine("BYöntemi çalışıyor...");
    }
}

```



```

    throw new InvalidOperationException("Derinlerde bir hata oluştu!");
    // Buradan sonra Stack Unwinding başlar:
    // 1. BYönteminden çıkılır.
    // 2. AYönteminde catch yok, oradan da çıkılır.
    // 3. Main'deki catch bloğuna ulaşılır.
}
}

```

12.9 Özel Durum İşleme Kuralları

Kontrol Akışını İstisna Kodları ile Gerçekleştirmeyin!

İstisnaları (özel durumları) işleyen kod ile iş mantığını içeren kod nesne yönelimli paradigmada her zaman ayrı olmalıdır! Bunun için;

- ◊ İş mantığını içeren kontrol akışı (**control flow**) istisna kodlarıyla yapılmamalıdır. Bunun yerine kontrol işlemleri (**control operations**) ile iş mantığı gerçekleştirilmelidir.
- ◊ Ayrıca Bir kontrol akışı, **if-else** ifadeleriyle açıkça yapılabiliyorsa, okunabilirliği ve performansı azalttığı için istisnalar kullanılmaz.

Aşağıdaki örneği göz önüne alalım;

```

object myObject;
void NesneYaz()
{
    Console.WriteLine(myObject.ToString()); //Bunu YAPMA!
}

```

Bu örnekle `Console.WriteLine` satırı icra edilmeye çalışıldığında `NullReferenceException` istisnası oluşur. Bunu yönetmek için aşağıdaki gibi bir kod yazmamalıyız;

```

// Bunu YAPMA!
object myObject;
void NesneYaz()
try
{
    Console.WriteLine(myObject.ToString());
}
catch(NullReferenceException ex)
{
    // eğer object'in yeni bir örneğini oluşturup myObject'e atarsam sorunu çözebilirim.
    myObject = new object();
    // artık myObject ile çalışmaya devam edebilirim
    NesneYaz();
}

```

Doğru olan kodlama aşağıdaki gibi olmalıdır;

```

// Bunu YAP!
object myObject;
void NesneYaz()
{
    if(myObject == null)
        myObject = new object();
    // Programın icrası bu noktaya ulaştığında, myObject'in null olmadığından emin oluruz
    NesneYaz();
}

```

Zorunlu Olmadıkça İstisnayı Tekrar İmal Etmeyin!

İstisnaları **tekrar fırlatmak** (**re-throw**) pahalıdır. Performansı olumsuz etkiler! Rutin olarak başarısız olan kodlar için, performans sorunlarını en aza indirmek için **tasarım desenlerini** (**design pattern**) kullanabilirsiniz. Microsoft sitesinde yer alan İstisnalar ve Performans konusu istisnaların performansı önemli ölçüde etkileyebileceği zamanlarda faydalı olan iki tasarım desenini açıklamaktadır.

İz Kaydı Olmadan İstisnaları Görmezden Gelmeyin!

Aşağıdaki gibi bir istisna işleme kötü bir uygulamadır;

```
try
{
    //...
}
catch(Exception ex) { } // Boş catch bloğu kötü uygulamadır.
```

İstisnaları görmezden gelmek o anı kurtaracaktır ancak daha sonra sürdürülebilirlik için bir kaos yaratacaktır. İstisnaların **iz kaydını (log)** günlüğe kaydederken, istisna örneğini her zaman kaydetmelisiniz. Böylece tüm yığın izi günlüğe kaydedilir ve yalnızca istisna mesajı kaydedilmez! Aşağıdaki gibi bir kod daha doğru olabilir;

```
try
{
    //...
}
catch(NullReferenceException ex)
    LogManager.Log(ex.ToString());
```

İşleyemeyeceğiniz İstisnaları Yakalamayın!

İşleyemeyeceğiniz istisnaları yakalamayın. Sadece yakalamayın. Yalnızca o konumda işleyebildiğiniz bir istisnayı yakalamalısınız. Bunun yerine, iyi bir çözümünüz olana veya ana fonksiyona ulaşana kadar bunların çağrı ağacında yukarı doğru ilerlemesine izin verin, bu durumda hata bilgilerinin doğru şekilde yönlendirildiğinden emin olabilirsiniz.

12.10 Düşün ve Çöz

- ♦ **Düşün:** İstisna yönetimi (**exception handling**) ile geleneksel hata kodu döndürme yöntemi arasındaki farkları karşılaştırınız. Hangi durumlarda istisna fırlatmak, hangi durumlarda **null** veya **bool** döndürmek daha uygundur?
- ♦ **Düşün:** **catch(Exception e)** bloğu ile tüm istisnaları yakalamak neden kötü bir pratik olarak değerlendirilir? 'Pokemon exception handling' ifadesi neyi eleştirir?
- ♦ **Düşün:** **finally** bloğu ne zaman çalışır? Bir veritabanı bağlantısını **finally** yerine **using** bloğuyla kapatmanın neden daha iyi bir tasarım olduğunu açıklayınız.
- ♦ **Düşün:** İstisnayı yeniden fırlatmak (**rethrow**) için **throw**; ile **throw ex**; arasındaki fark nedir? Bu farkın hata ayıklama (**debugging**) sürecindeki önemi nedir?
- ♦ **Düşün:** Kullanıcı tanımlı istisna (**custom exception**) ne zaman yazılmalıdır? **Exception** sınıfından mı, **ApplicationException**'dan mı türetilmelidir? Bir stok yönetim sisteminde hangi özel istisnalar tanımlamak mantıklı olur?
- ♦ **Düşün:** İstisna hiyerarşisi (**Exception** → **SystemException** → **IOException** gibi) **catch** bloklarını nasıl etkiler? Daha spesifik istisnaları daha üstteki **catch** bloklarına yazmak neden derleme hatasına yol açar?
- ♦ **Düşün:** **try-catch** bloklarının performans maliyeti var mıdır? İstisnaların normal akış kontrolü için (örneğin **int.Parse** yerine **try-catch**) kullanılması neden önerilmez?
- ♦ **Çöz:** Kullanıcıdan iki sayı ve bir işlem (+, -, *, /) alarak sonucu hesaplayan; sıfıra bölme, geçersiz işlem ve sayısal olmayan giriş durumlarını ayrı **catch** bloklarıyla ele alan bir hesap makinesi programı yazınız.
- ♦ **Çöz:** **YetersizBakiyeException** ve **HesapKapaliException** adında iki özel istisna sınıfı oluşturun. Bunları bir banka hesabı sınıfında uygun yerlerde fırlatın ve test edin.
- ♦ **Çöz:** Bir dosyadan satır satır sayı okuyan ve toplayan program yazınız. Dosya bulunamazsa, satır sayısal değilse veya beklenmedik bir hata oluşursa her durum için ayrı hata mesajı gösterilsin. **finally** bloğuyla dosya kapatma işlemini güvence altına alın.
- ♦ **Çöz:** **AggregateException** kullanarak birden fazla hatayı tek bir istisna nesnesinde toplayan bir senaryo kurunuz. Bu yapının paralel programlamada neden kullanıldığını açıklayınız.
- ♦ **Çöz:** İstisna filtreleri (**exception filters**, **when** koşulu) kullanarak HTTP durum koduna göre farklı davranış sergileyen bir örnek yazınız.